
TRƯỜNG ĐẠI HỌC PHENIKAA
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

***XÂY DỰNG ỨNG DỤNG QUẢN LÝ NHÂN SỰ ĐẠI HỌC
PHENIKAA***

Lớp: Lập trình hướng đối tượng-1-2-25(N01)

Giảng viên hướng dẫn: TS. Trần Đăng Hoan

Sinh viên thực hiện:

<i>Họ và tên</i>	<i>Mã số sinh viên</i>
<i>Dương Công Minh</i>	<i>22010009</i>

HÀ NỘI, THÁNG 03/2026

Mục Lục

Mục Lục.....	1
Lời mở đầu	2
1. Mục tiêu	3
2. Phân tích yêu cầu	3
2.1 Sơ đồ ca sử dụng (Use Case Diagram)	3
2.2 Giao diện cửa sổ với Swing.....	4
2.3 Sơ đồ class (Thiết kế kiến trúc MVC)	4
2.4 Các tính năng và chức năng trong hệ thống	5
2.5 Dữ liệu được lưu trữ dưới dạng Collection	6
2.6 Lưu dữ liệu xuống file nhị phân	6
2.7 Kiểm tra dữ liệu khi nhập vào	6
3. Thiết kế và triển khai.....	6
3.1 Giao diện đăng nhập (class "Login").....	6
3.2 Giao diện hệ thống QLNS (class "MainMenu").....	8
3.3 Giao diện hệ thống QLNS (class "HRManagementGUI")	9
3.3.1 Các tính năng hệ thống.....	10
3.3.2 Các chức năng nghiệp vụ	14
4. Kết luận	18
4.1 Đánh giá kết quả đạt được	18
4.2 Hướng phát triển của phần mềm.....	18
Tổng kết	20
Tài liệu tham khảo.....	21
Phụ lục: Source Code	22
A. Khối MODEL (Các lớp thực thể và dữ liệu)	22
B. Khối VIEW - Giao diện Đăng nhập	26
C. Khối CONTROLLER - Xử lý nghiệp vụ	46

Lời mở đầu

Trong thời đại công nghệ số hiện nay, việc ứng dụng Công nghệ thông tin vào quản lý hành chính và nhân sự là yêu cầu tất yếu đối với mọi tổ chức. Đặc biệt, tại Trường Đại học Phenikaa – một môi trường giáo dục đại học đổi mới sáng tạo, với quy mô nhân sự ngày càng mở rộng bao gồm hàng ngàn Giảng viên, Nhà khoa học và Cán bộ nhân viên hành chính, việc quản lý nhân sự bằng phương pháp thủ công hoặc Excel đã không còn đáp ứng đủ nhu cầu về tính bảo mật, chính xác và tốc độ.

Hệ thống quản lý nhân sự tại trường Đại học mang những đặc thù riêng biệt so với các doanh nghiệp thông thường. Ngoài việc quản lý thông tin cơ bản và lương cứng, hệ thống còn phải xử lý các bài toán phức tạp như: phụ cấp thâm niên, phụ cấp học hàm/học vị (Tiến sĩ, Thạc sĩ), số tiết giảng dạy và đặc biệt là chính sách khen thưởng năng lực Nghiên cứu khoa học (bài báo quốc tế ISI/Scopus) – một trong những định hướng mũi nhọn của Đại học Phenikaa.

Xuất phát từ thực tiễn đó, trong khuôn khổ môn học Lập trình hướng đối tượng, nhóm chúng tôi với đề tài: "Xây dựng phần mềm Quản lý Nhân sự Đại học Phenikaa" áp dụng triệt để các kiến thức về Lập trình hướng đối tượng (OOP), giao diện đồ họa (Java Swing), và xử lý luồng dữ liệu (File I/O) để giải quyết một bài toán thực tế mang tính ứng dụng cao.

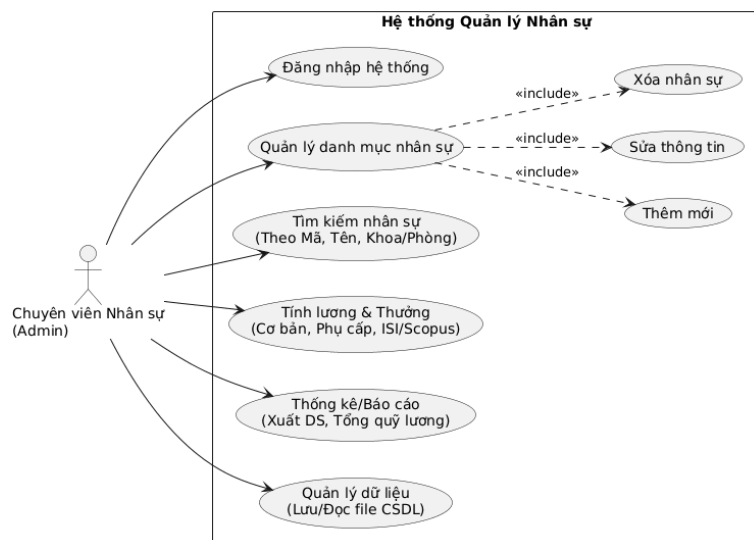
1. Mục tiêu

Mục tiêu cốt lõi của bài tập lớn này là thiết kế và xây dựng thành công một ứng dụng Desktop hoàn chỉnh bằng ngôn ngữ lập trình Java, phục vụ trực tiếp cho bài toán quản lý cán bộ, giảng viên tại Đại học Phenikaa. Về mặt học thuật, dự án yêu cầu sinh viên phải áp dụng triệt để bốn tính chất nền tảng của lập trình hướng đối tượng bao gồm tính Đóng gói, Kế thừa, Đa hình và Trừu tượng. Về mặt kỹ thuật phần mềm, hệ thống hướng tới việc xây dựng giao diện người dùng thân thiện thông qua thư viện Java Swing, tổ chức mã nguồn khoa học theo mô hình MVC (Model - View - Controller), xử lý lưu trữ dữ liệu an toàn với cơ chế đọc ghi file nhị phân và đảm bảo tính toàn vẹn của dữ liệu thông qua các kỹ thuật xử lý ngoại lệ tiên tiến. Thành phẩm cuối cùng là một phần mềm có khả năng vận hành ổn định, phản hồi nhanh chóng các thao tác nghiệp vụ của người dùng.

2. Phân tích yêu cầu

2.1 Sơ đồ ca sử dụng (Use Case Diagram)

Để hệ thống hóa các chức năng nghiệp vụ trước khi tiến hành viết mã, sơ đồ ca sử dụng được thiết lập nhằm mô tả trực quan sự tương tác giữa người dùng và phần mềm. Trong bối cảnh quản lý nhân sự tại Đại học Phenikaa, tác nhân chính (Actor) vận hành hệ thống là Chuyên viên Phòng Tổ chức Nhân sự. Tác nhân này thực hiện một chuỗi các ca sử dụng cốt lõi bao gồm: đăng nhập vào hệ thống bảo mật, quản lý danh mục hồ sơ (thêm mới, sửa đổi, xóa thông tin cán bộ), tìm kiếm nhân sự theo các tiêu chí định danh, tự động hóa việc tính toán lương thưởng và kết xuất các báo cáo thống kê tổng quỹ lương. Sơ đồ này đóng vai trò như một bản thiết kế chức năng, định hướng rõ ràng cho toàn bộ quá trình phát triển giao diện và logic mã nguồn ở các giai đoạn tiếp theo.



2.2 Giao diện cửa sổ với Swing

Để đảm bảo trải nghiệm người dùng tối ưu và phù hợp với nghiệp vụ hành chính văn phòng, hệ thống được yêu cầu thiết kế dưới dạng ứng dụng có giao diện đồ họa (GUI) thay vì chạy trên màn hình Console truyền thống. Thư viện Java Swing được lựa chọn để triển khai lớp View. Giao diện cần được thiết kế mang tính chuyên nghiệp, chia thành các phân vùng rõ ràng như khu vực nhập liệu, khu vực bảng hiển thị danh sách (sử dụng JTable) và khu vực các nút chức năng thao tác (sử dụng JButton). Các thành phần đồ họa này giúp người quản trị dễ dàng theo dõi toàn cảnh bức tranh nhân sự của nhà trường và thực hiện các thao tác một cách trực quan nhất.

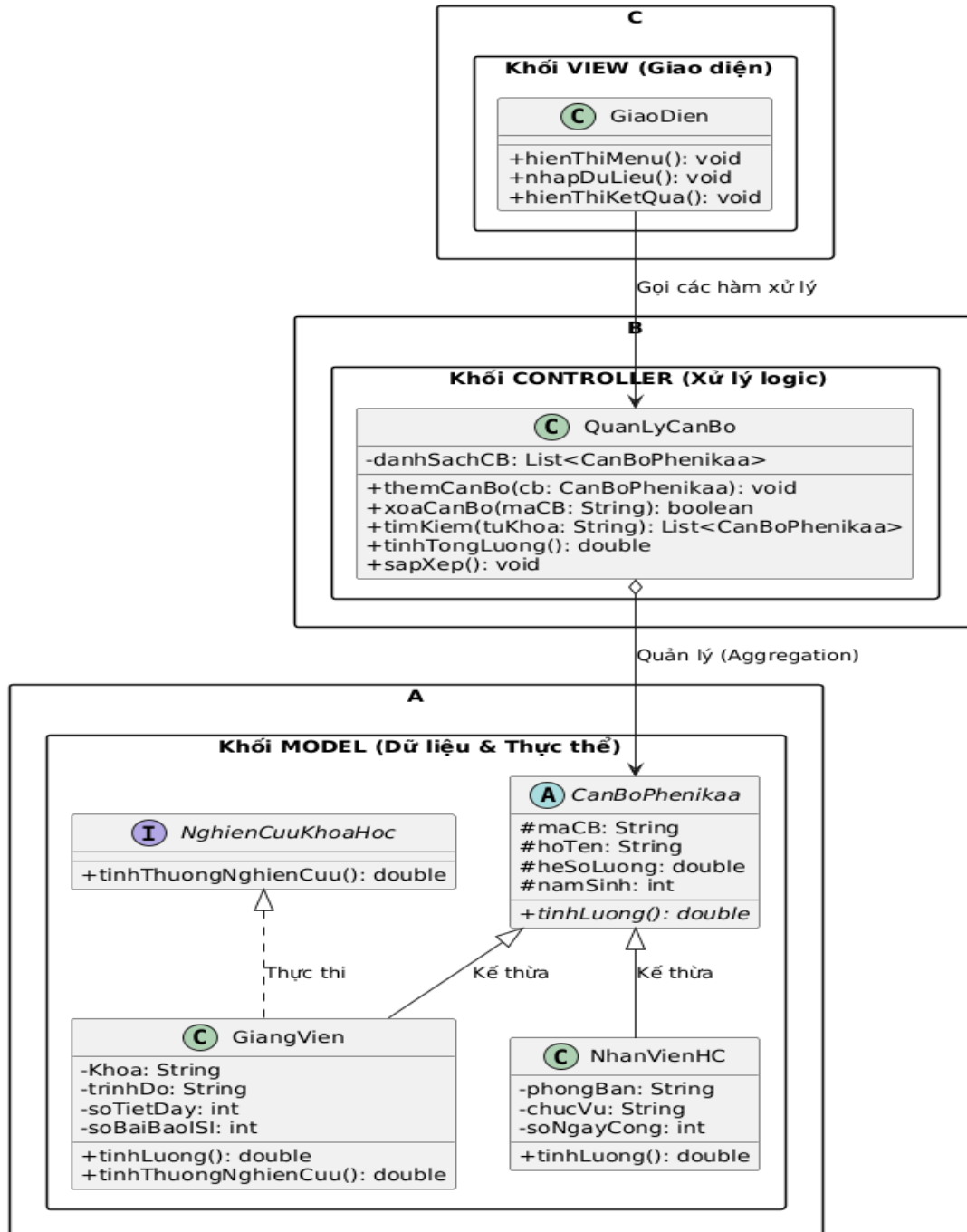
2.3 Sơ đồ class (Thiết kế kiến trúc MVC)

Kiến trúc cốt lõi của hệ thống phần mềm được thiết kế tuân thủ nghiêm ngặt mô hình MVC (Model - View - Controller), giúp phân tách rõ ràng giữa cấu trúc dữ liệu, nghiệp vụ xử lý và giao diện tương tác.

Đối với Khối MODEL (Khối A - Dữ liệu và Thực thể), toàn bộ cán bộ, nhân viên được khái quát hóa thông qua một Abstract Class mang tên CanBoPhenikaa. Lớp này đại diện cho thông tin chung cơ bản của mọi cán bộ trong trường như mã định danh, họ tên, năm sinh, hệ số lương, đồng thời định nghĩa phương thức trừu tượng dùng để tính lương. Từ lớp cơ sở này, hệ thống mở rộng thành Class NhanVienHC đại diện cho thực thể nhân viên hành chính với các thuộc tính chuyên biệt gồm phòng ban, chức vụ, số ngày công và tự định nghĩa cách tính lương đặc thù theo ngày công. Song song đó, Class GiangVien đại diện cho thực thể giảng viên, chứa các thông tin đặc thù của khối học thuật như khoa trực thuộc, trình độ, số tiết giảng dạy và số bài báo ISI. Đặc biệt, nhằm số hóa chính sách trọng dụng nhân tài của Đại học Phenikaa, một Interface mang tên NghienCuuKhoaHoc được thiết kế độc lập nhằm định nghĩa hành vi tính thưởng nghiên cứu khoa học, và lớp giảng viên sẽ thực thi (implements) giao diện này.

Đối với Khối CONTROLLER (Khối B - Xử lý logic), Class QuanLyCanBo đóng vai trò là bộ điều khiển trung tâm. Lớp này sử dụng cấu trúc Collection để chứa và quản lý danh sách toàn bộ cán bộ. Nhiệm vụ cốt lõi của QuanLyCanBo là chịu trách nhiệm xử lý toàn bộ các nghiệp vụ logic của bài toán bao gồm: thêm hồ sơ, xóa hồ sơ, tìm kiếm nhân sự, tính tổng quỹ lương toàn trường và sắp xếp danh sách tối ưu.

Cuối cùng, đối với Khối VIEW (Khối C - Giao diện), hệ thống hiện thực hóa qua Class GiaoDien. Lớp này đại diện cho bề mặt tương tác trực tiếp với người dùng, đảm nhiệm duy nhất vai trò hiển thị các thanh menu, tiếp nhận dữ liệu đầu vào từ bàn phím hoặc chuột, và in kết quả trả về ra màn hình giao diện. Việc tách bạch Class GiaoDien giúp mã nguồn dễ dàng bảo trì và không làm ảnh hưởng đến luồng dữ liệu bên trong.



2.4 Các tính năng và chức năng trong hệ thống

Phần mềm cung cấp một hệ sinh thái các chức năng khép kín để quản lý vòng đời dữ liệu nhân sự. Hệ thống cho phép người dùng thực hiện các thao tác thêm mới hồ sơ khi có cán bộ trúng tuyển, cập nhật thông tin khi có sự thay đổi về chức vụ hoặc học hàm, và xóa hồ sơ đối với các trường hợp chuyển công tác. Bên cạnh đó, tính năng tìm kiếm thông minh được tích hợp, hỗ trợ truy xuất nhanh chóng một hồ sơ dựa trên mã định danh hoặc gần

đúng theo tên gọi. Chức năng quan trọng nhất của phần mềm là cơ chế tự động tính lương linh hoạt nhờ vào tính Đa hình của OOP; hệ thống có khả năng tự nhận diện đối tượng là Giảng viên hay Nhân viên để áp dụng công thức tính toán lương, phụ cấp và tiền thưởng nghiên cứu khoa học tương ứng, từ đó xuất ra báo cáo tổng quỹ lương của toàn trường một cách chính xác tuyệt đối.

2.5 Dữ liệu được lưu trữ dưới dạng Collection

Nhằm xử lý linh hoạt khối lượng dữ liệu biến động liên tục trong quá trình chạy chương trình, cấu trúc dữ liệu ArrayList thuộc bộ thư viện Collections Framework của Java được áp dụng làm bộ nhớ đệm chính. Việc sử dụng Collection không chỉ khắc phục được nhược điểm giới hạn kích thước cố định của mảng thông thường mà còn cung cấp sẵn các phương thức tối ưu để sắp xếp danh sách nhân sự theo tên hoặc theo mức lương, đồng thời tăng tốc độ thêm, sửa, xóa các phần tử đối tượng trong bộ nhớ tạm thời của hệ thống.

2.6 Lưu dữ liệu xuống file nhị phân

Để đảm bảo dữ liệu không bị thất thoát sau khi tắt ứng dụng, hệ thống yêu cầu cơ chế lưu trữ lâu dài (Persistence). Cơ chế tuần tự hóa đối tượng (Object Serialization) được sử dụng để ghi toàn bộ danh sách Collection chứa các đối tượng cán bộ xuống một tệp tin nhị phân (có đuôi .dat). Khi khởi động lại phần mềm, tiến trình giải tuần tự hóa (Deserialization) sẽ đọc luồng byte từ file này và phục hồi nguyên trạng danh sách nhân sự lên bộ nhớ và hiển thị lên giao diện JTable. Phương pháp này giúp bảo mật dữ liệu tốt hơn so với file văn bản thuần túy và giữ nguyên được cấu trúc phức tạp của các lớp đối tượng.

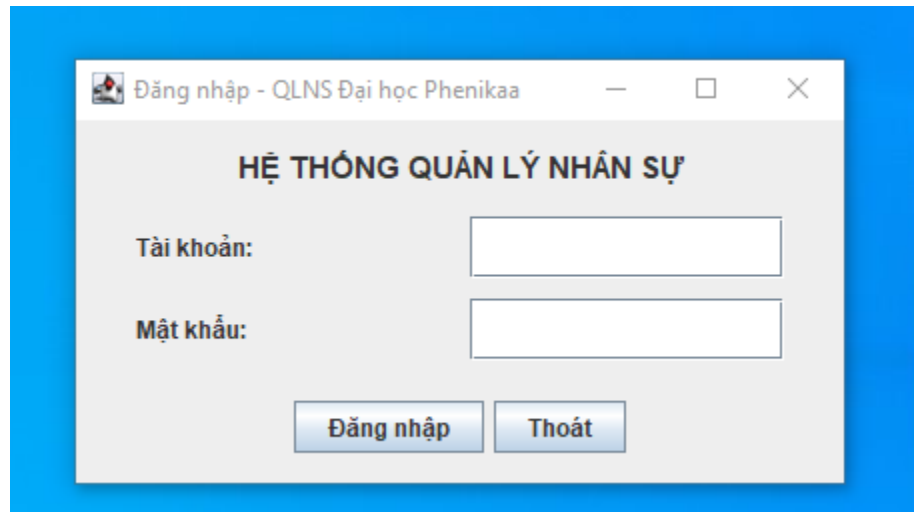
2.7 Kiểm tra dữ liệu khi nhập vào

Tính toàn vẹn và hợp lệ của dữ liệu đầu vào là yếu tố sống còn của một phần mềm quản lý. Hệ thống được trang bị các cơ chế kiểm duyệt chặt chẽ (Validation) tại giao diện nhập liệu. Cụ thể, hệ thống sẽ phát cảnh báo nếu người dùng bỏ trống các trường thông tin bắt buộc, ngăn chặn việc nhập trùng lặp Mã cán bộ đã tồn tại. Đối với các dữ liệu dạng số như năm sinh, số ngày công hay số bài báo khoa học, các khối lệnh try-catch được triển khai tinh tế để bắt các lỗi định dạng (NumberFormatException), đảm bảo không cho phép nhập chữ cái hoặc số âm vào các trường này. Nhờ vậy, phần mềm duy trì được sự ổn định, tránh nguy cơ treo ứng dụng do sự sơ suất của người nhập liệu.

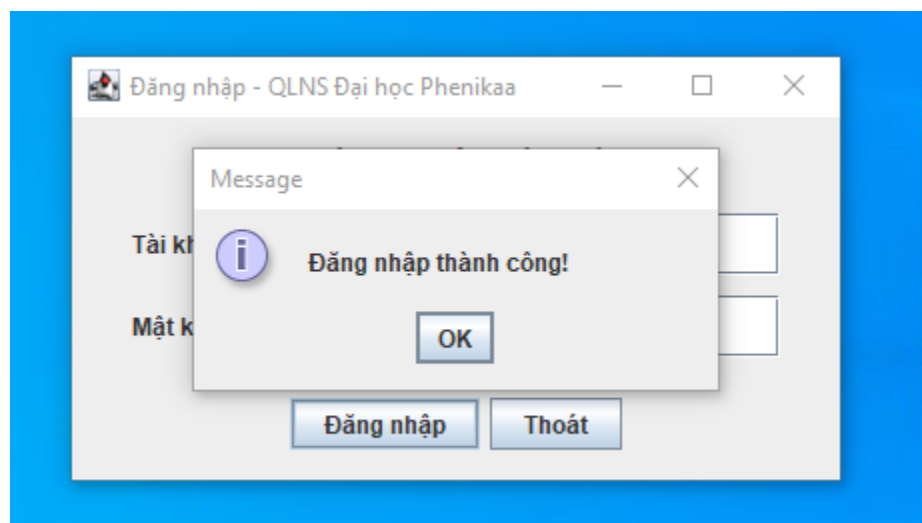
3. Thiết kế và triển khai

3.1 Giao diện đăng nhập (class "Login")

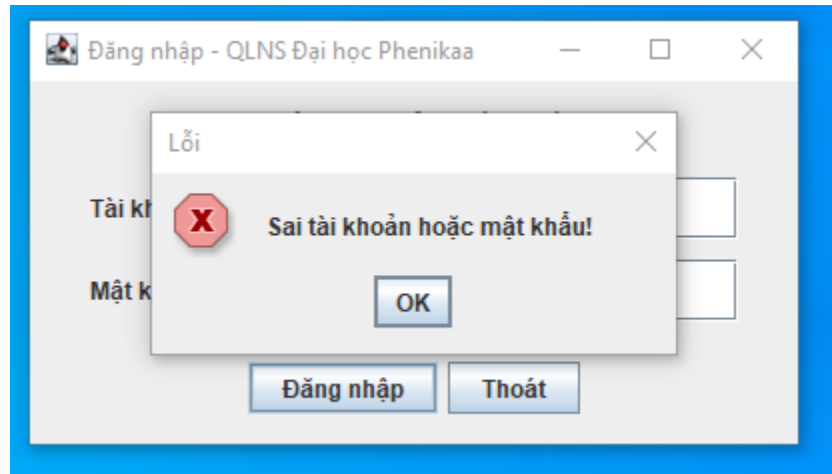
Bảo mật thông tin là yếu tố tiên quyết đối với bất kỳ hệ thống quản trị nhân sự nào, đặc biệt là với dữ liệu nhạy cảm như lương thưởng và hồ sơ cán bộ của Đại học Phenikaa. Do đó, lớp giao diện Login được thiết kế đóng vai trò như một "người gác cổng", yêu cầu người dùng phải xác thực danh tính trước khi truy cập vào hệ thống chính. Giao diện này được xây dựng trên nền tảng JFrame của thư viện Swing, bao gồm các thành phần cơ bản như trường nhập tài khoản (JTextField), trường nhập mật khẩu được mã hóa ký tự (JPasswordField) nhằm bảo vệ tầm nhìn, cùng với các nút lệnh xác nhận và thoát chương trình.



Về mặt luồng sự kiện (Event Handling), khi chuyên viên nhân sự nhấn nút "Đăng nhập", hệ thống sẽ kích hoạt bộ lắng nghe sự kiện (ActionListener) để thu thập chuỗi ký tự vừa nhập. Dữ liệu này sau đó được đối chiếu với thông tin tài khoản quản trị đã được thiết lập sẵn hoặc lưu trữ trong cơ sở dữ liệu. Nếu thông tin khớp lệnh, cửa sổ Login sẽ tự động đóng lại, đồng thời khởi tạo và gọi hiển thị lớp giao diện chính của phần mềm.



Ngược lại, nếu người dùng nhập sai, một hộp thoại cảnh báo (JOptionPane) sẽ xuất hiện, yêu cầu kiểm tra lại thông tin, qua đó ngăn chặn tuyệt đối các hành vi truy cập trái phép vào kho dữ liệu nhân sự của nhà trường.



3.2 Giao diện hệ thống QLNS (class "MainMenu")

Để tối ưu hóa không gian làm việc và chuyên môn hóa nghiệp vụ cho chuyên viên nhân sự, giao diện được thiết kế theo kiến trúc phân hệ (Modular Architecture) thông qua thành phần JTabbedPane của thư viện Swing. Màn hình chính được chia thành các Thẻ (Tab) độc lập: "Phân hệ Giảng viên" và "Phân hệ Hành chính".

Mỗi phân hệ hoạt động như một không gian làm việc khép kín, sở hữu riêng biểu mẫu nhập liệu đặc thù, hệ thống nút bấm điều khiển và một bảng dữ liệu (JTable) chỉ hiển thị danh sách của nhóm đối tượng tương ứng. Thiết kế này mang lại sự mạch lạc, ngăn chặn rủi ro nhập liệu chéo và thể hiện phong cách của các phần mềm quản trị doanh nghiệp chuyên nghiệp.



3.3 Giao diện hệ thống QLNS (class "HRManagementGUI")

Lớp HRManagementGUI là trung tâm tương tác chính của toàn bộ phần mềm, nơi thể hiện rõ nét nhất kiến trúc View trong mô hình MVC. Để tối ưu hóa không gian làm việc cho chuyên viên nhân sự, giao diện được chia bố cục một cách khoa học bằng các Layout Manager (như BorderLayout kết hợp GridLayout). Không gian màn hình được chia thành ba khu vực trọng tâm: Khu vực nhập liệu nằm ở phía trên hoặc bên trái chứa các biểu mẫu thông tin; Khu vực điều khiển chứa hệ thống nút bấm nghiệp vụ; và Khu vực hiển thị trung tâm sử dụng thành phần JTable để trực quan hóa danh sách giảng viên.

Quản lý Giảng viên - Đại học Phenikaa

Thông tin chi tiết hồ sơ nhân sự

Mã Cán bộ: Họ và Tên:

Năm sinh: Hệ số lương:

Loại cán bộ:

Khoa công tác: Trình độ:

Số tiết dạy: Số bài ISI:

Danh sách Giảng viên

Mã CB	Họ Tên	Năm sinh	Phân loại	Đơn vị/Phòng	Tổng Lương (VNĐ)
GV001	Nguyễn Văn An	1980	Giảng viên	Khoa: CNTT	38.700.000
GV002	Trần Thị Bích	1985	Giảng viên	Khoa: Kinh Tế	20.640.000

Quay lại Menu Thêm mới Cập nhật Xóa Tìm kiếm Sắp xếp lương Nhập File Xuất File Thống kê Làm mới

hoặc cán bộ dưới dạng lưới dữ liệu, giúp người dùng dễ dàng bao quát toàn bộ thông tin tùy chọn vào việc chọn phân hệ quản lý nào

Quản lý Nhân viên Hành chính - Đại học Phenikaa

Thông tin chi tiết hồ sơ nhân sự

Mã Cán bộ: Họ và Tên:

Năm sinh: Hệ số lương:

Loại cán bộ:

Phòng ban: Chức vụ:

Số ngày công:

Danh sách Nhân viên Hành chính

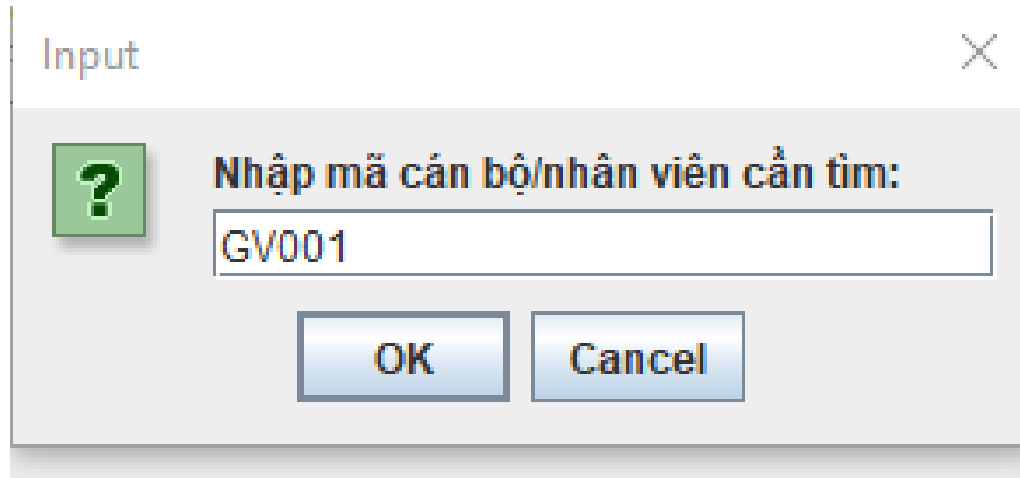
Mã CB	Họ Tên	Năm sinh	Phân loại	Đơn vị/Phòng	Tổng Lương (VNĐ)
NV001	Lê Hoàng	1990	Nhân viên HC	Phòng: Phòng Đào tạo	11.212.000
NV002	Phạm Văn Dũng	1978	Nhân viên HC	Phòng: Phòng Tổ chức	17.700.000

Quay lại Menu Thêm mới Cập nhật Xóa Tìm kiếm Sắp xếp lương Nhập File Xuất File Thống kê Làm mới

3.3.1 Các tính năng hệ thống

vào ô tìm kiếm hệ thống sẽ lập tức duyệt qua cấu trúc Collection, lọc ra các hồ sơ trùng khớp và làm mới (refresh) lại bảng hiển thị chỉ với những kết quả vừa tìm được. Tính năng này tối ưu hóa đáng kể thời gian truy xuất thông tin hồ sơ của bất kỳ cá nhân nào trong trường.

Khi tìm kiếm



Sau khi tìm kiếm

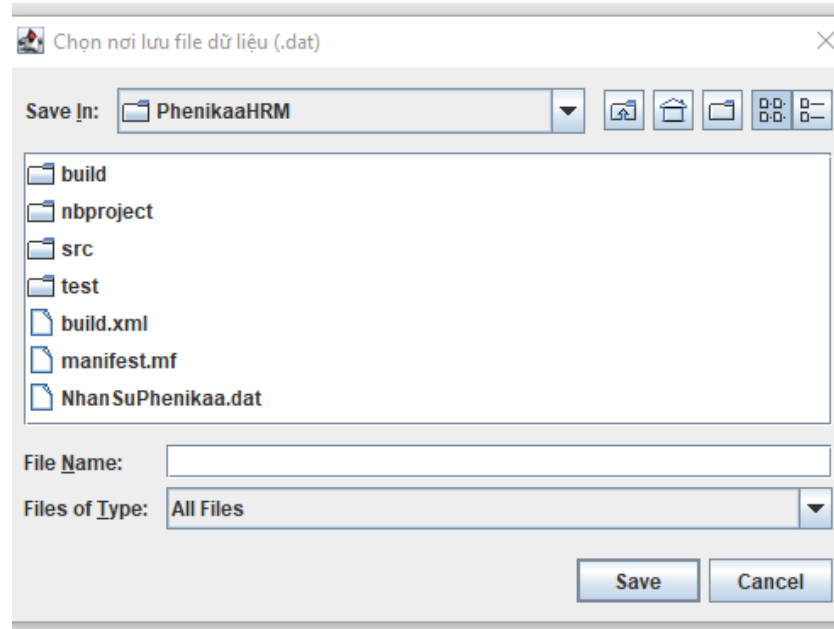
Danh sách Nhân sự					
Mã CB	Họ Tên	Năm sinh	Phân loại	Đơn vị/Phòng	Tổng Lương (VNĐ)
GV001	Nguyễn Văn An	1980	Giảng viên	Khoa: CNTT	38.700.000

Phần tìm kiếm nhân sự của phân hệ nhân viên cũng tương tự

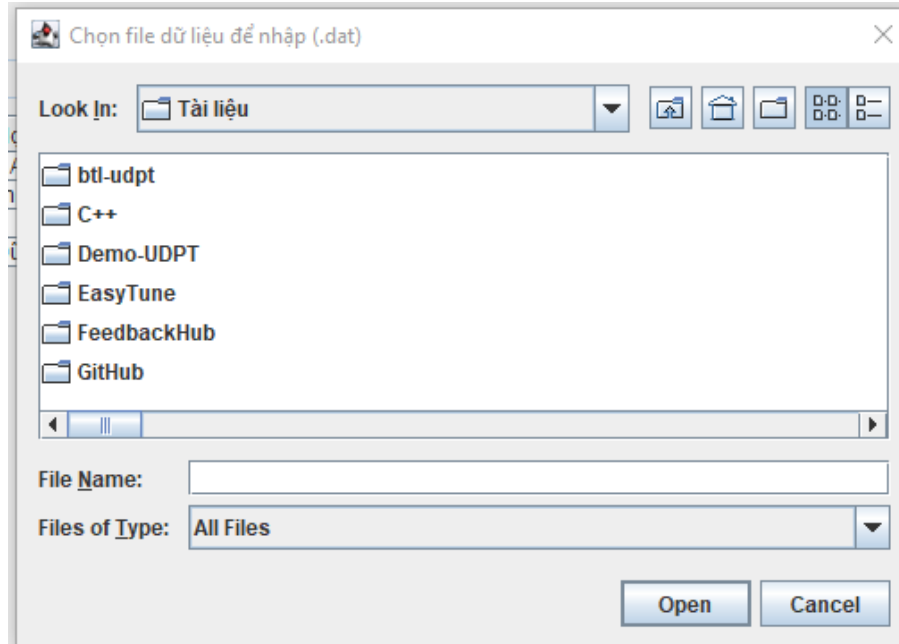
c) Lưu thông tin và Tải file lên

Để đảm bảo tính toàn vẹn và lưu trữ dữ liệu lâu dài, hệ thống cung cấp bộ đôi tính năng Xuất File (Lưu thông tin) và Nhập File (Tải dữ liệu). Điểm nổi bật của chức năng này là việc tích hợp hộp thoại tương tác JFileChooser của thư viện Swing, mang lại trải nghiệm linh hoạt cho người dùng.

Cụ thể, chức năng Xuất File cho phép chuyên viên nhân sự chủ động chọn thư mục đích trên máy tính để xuất toàn bộ danh sách hiện tại từ bộ nhớ đệm xuống một tệp tin nhị phân (định dạng .dat). Quá trình này được thực hiện thông qua cơ chế Object Serialization của Java, đảm bảo dữ liệu cấu trúc phức tạp không bị mất mát khi kết thúc phiên làm việc.



Ngược lại, tính năng Nhập File cho phép người dùng duyệt tìm và chọn tệp dữ liệu đã lưu trữ từ trước. Khi tệp được tải lên, hệ thống sẽ tự động kích hoạt quá trình Deserialization, tiến hành đọc luồng byte từ tệp tin và khôi phục lại nguyên trạng danh sách nhân sự lên bảng dữ liệu (JTable). Cơ chế này không chỉ giúp sao lưu an toàn mà còn tạo sự liên mạch hoàn hảo cho các phiên làm việc tiếp theo.

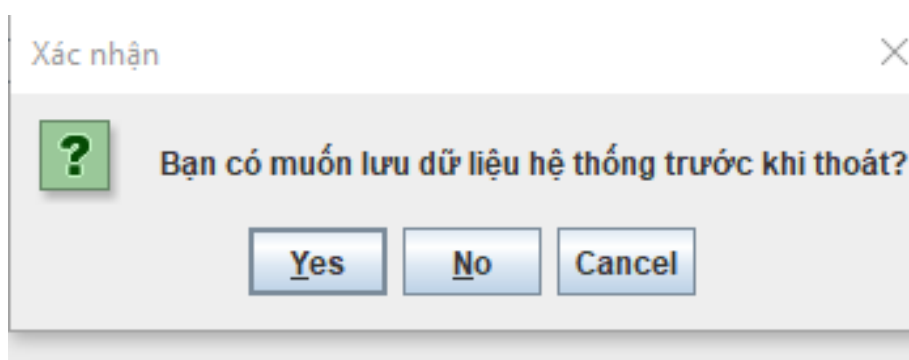


d) Tải lại giao diện

Bên cạnh đó, nhằm duy trì tính ổn định của giao diện trong quá trình thao tác liên tục, tính năng Tải lại giao diện (Refresh/Clear Form) được thiết kế để hỗ trợ người dùng làm sạch nhanh các trường nhập liệu. Chỉ với một nút bấm, toàn bộ các ô nhập văn bản (JTextField) và hộp thoại thả xuống (JComboBox) sẽ được khôi phục về trạng thái trống ban đầu, chuẩn bị sẵn sàng cho một chu kỳ nhập liệu mới mà không bị lẫn lộn dữ liệu cũ.

e) Thoát ứng dụng

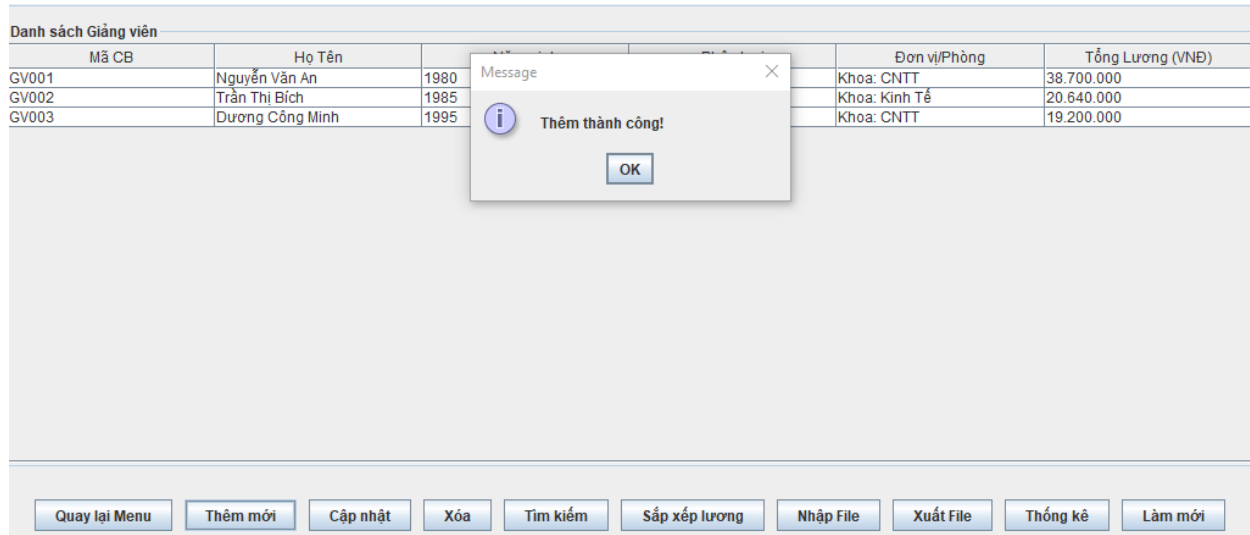
Cuối cùng, tính năng **Thoát ứng dụng** được lập trình đi kèm với một hộp thoại xác nhận (Confirmation Dialog) trước khi đóng hoàn toàn chương trình, nhằm nhắc nhở người dùng lưu lại các thay đổi mới nhất (nếu có) trước khi ngắt kết nối hệ thống.



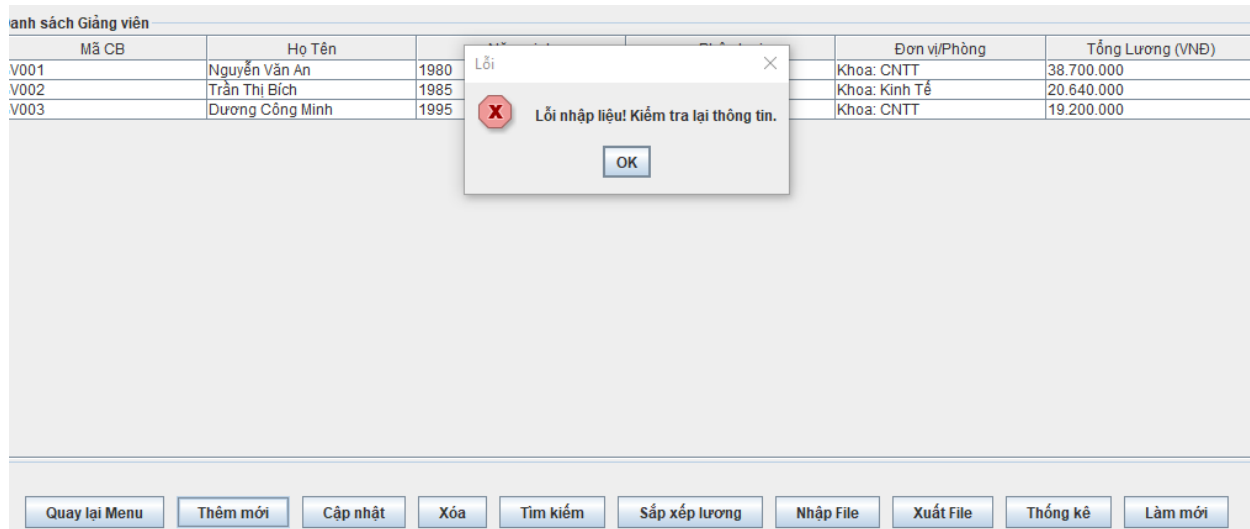
3.3.2 Các chức năng nghiệp vụ

a) Thêm nhân sự

Nắm vai trò cốt lõi trong vòng đời quản lý dữ liệu (CRUD), chức năng **Thêm nhân sự** là điểm khởi đầu của mọi quy trình nghiệp vụ. Khi chuyên viên nhân sự điền đầy đủ các thông tin vào biểu mẫu và xác nhận, hệ thống giao diện sẽ tiến hành bóc tách dữ liệu. Dựa trên lựa chọn loại đối tượng là "Giảng viên" hay "Nhân viên hành chính", hệ thống sẽ khởi tạo một thực thể tương ứng (Object) mang đặc thù riêng biệt (như số bài báo ISI đối với giảng viên hoặc số ngày công đối với nhân viên). Đối tượng này sau đó được đẩy vào bộ nhớ danh sách (Collection) và ngay lập tức hiển thị lên bảng dữ liệu tổng hợp.



Nếu dữ liệu nhập vào không hợp lệ, sau khi chọn nút “Thêm” class sẽ hiển thị hộp thông báo lỗi.



b) **Chỉnh sửa thông tin nhân sự**

Trong suốt quá trình công tác tại Đại học Phenikaa, hồ sơ cán bộ thường xuyên có sự biến động. Do đó, chức năng **Chỉnh sửa thông tin nhân sự** được thiết kế vô cùng linh hoạt. Người dùng chỉ cần thao tác nhấp chuột trực tiếp vào một dòng hồ sơ bất kỳ trên bảng JTable, toàn bộ thông tin của cá nhân đó sẽ tự động được đẩy ngược (binding) lên các ô nhập liệu phía trên. Từ đây, chuyên viên có thể dễ dàng cập nhật các thay đổi như thăng hạng chức danh, tăng hệ số lương hoặc bổ sung số lượng bài báo nghiên cứu khoa học mới đạt chuẩn quốc tế. Khi nhấn xác nhận cập nhật, hệ thống sẽ ghi đè thông tin mới lên đối tượng cũ trong bộ nhớ và tự động làm mới bảng hiển thị.

Quản lý Giảng viên - Đại học Phenikaa

Thông tin chi tiết hồ sơ nhân sự

Mã Cán bộ: GV002 Họ và Tên: Trần Thị Bích

Năm sinh: 1987 Hệ số lương: 2.8

Loại cán bộ: Giảng viên

Khoa công tác: Kinh Tế Trình độ: Thạc sĩ

Số tiết dạy: 80 Số bài ISI: 1

Danh sách Giảng viên

Mã CB	Họ Tên	Đơn vị/Phòng	Tổng Lương (VNĐ)
GV001	Nguyễn Văn An	Khoa: CNTT	38.700.000
GV002	Trần Thị Bích	Khoa: Kinh Tế	20.640.000
GV003	Dương Công Minh	Khoa: CNTT	19.200.000

Message

Cập nhật thành công!

OK

Quay lại Menu Thêm mới Cập nhật Xóa Tìm kiếm Sắp xếp lương Nhập File Xuất File Thống kê Làm mới

c) Xóa nhân sự

Đi kèm với việc thêm và sửa, chức năng **Xóa nhân sự** được cung cấp để xử lý các trường hợp cán bộ chuyển công tác hoặc nghỉ hưu. Thao tác xóa được thực hiện thông qua việc chọn hồ sơ trên bảng dữ liệu. Tuy nhiên, để ngăn chặn rủi ro xóa nhầm do sơ suất, hệ thống luôn yêu cầu một bước xác nhận bảo mật (Pop-up Warning) trước khi chính thức gỡ bỏ đối tượng đó ra khỏi cấu trúc Collection và cập nhật lại giao diện tổng thể.

anh sách Giảng viên

Mã CB	Họ Tên	Đơn vị/Phòng	Tổng Lương (VNĐ)
V001	Nguyễn Văn An	Khoa: CNTT	38.700.000
V002	Trần Thị Bích	Khoa: Kinh Tế	20.640.000
V003	Dương Công Minh	Khoa: CNTT	19.200.000

Xác nhận

Chắc chắn xóa?

Yes No

Quay lại Menu Thêm mới Cập nhật Xóa Tìm kiếm Sắp xếp lương Nhập File Xuất File Thống kê Làm mới

d) Thống kê danh sách nhân sự

Điểm nhấn quan trọng nhất thể hiện sức mạnh của ứng dụng là chức năng Thống kê danh sách nhân sự và tự động tính quỹ lương. Chức năng này ứng dụng triệt để tính Đa hình (Polymorphism) trong lập trình hướng đối tượng. Khi kích hoạt, hệ thống sẽ duyệt qua toàn bộ danh sách, tự động nhận diện từng cá nhân là giảng viên hay nhân viên hành chính để gọi đúng công thức tính toán mức thu nhập (bao gồm lương cơ bản, các khoản phụ cấp thâm niên, phụ cấp chức vụ lãnh đạo và cộng dồn tiền thưởng cho các công trình nghiên cứu khoa học). Kết quả cuối cùng không chỉ hiển thị chi tiết trên từng dòng của bảng dữ liệu mà còn tổng hợp thành một báo cáo tổng quỹ lương nhà trường cần chi trả trong tháng, cung cấp số liệu cực kỳ trực quan cho Ban Giám hiệu và Phòng Tài chính.

Danh sách Giảng viên

Mã CB	Họ Tên	Đơn vị/Phòng	Tổng Lương (VNĐ)
GV001	Nguyễn Văn An	Khoa: CNTT	38.700.000
GV002	Trần Thị Bích	Khoa: Kinh Tế	20.640.000
GV003	Dương Công Minh	Khoa: CNTT	19.200.000

Thống kê

Tổng quỹ lương khối Giảng viên: 78.540.000 VNĐ

OK

Quay lại Menu Thêm mới Cập nhật Xóa Tìm kiếm Sắp xếp lương Nhập File Xuất File Thống kê Làm mới

4. Kết luận

4.1 Đánh giá kết quả đạt được

Trải qua quá trình phân tích, thiết kế và trực tiếp lập trình, bài tập lớn đã hoàn thành xuất sắc mục tiêu ban đầu đề ra: xây dựng thành công phần mềm Quản lý Nhân sự dành riêng cho đặc thù của Đại học Phenikaa. Về mặt học thuật, đồ án đã chứng minh được sự hiểu biết sâu sắc và khả năng vận dụng linh hoạt bốn tính chất trụ cột của Lập trình hướng đối tượng (OOP). Cụ thể, hệ thống đã đóng gói (Encapsulation) an toàn các dữ liệu nhạy cảm, áp dụng tính kế thừa (Inheritance) để tối ưu hóa việc quản lý các thực thể Giảng viên và Nhân viên hành chính từ một gốc cán bộ chung, đồng thời sử dụng tính đa hình (Polymorphism) để tự động hóa nghiệp vụ tính lương phức tạp. Đặc biệt, việc tích hợp thành công Interface NghienCuuKhoaHoc để xử lý chính sách khen thưởng các bài báo ISI/Scopus đã phản ánh đúng triết lý đổi mới sáng tạo và khuyến khích nghiên cứu của nhà trường.

Về mặt kỹ thuật phần mềm, ứng dụng đã được thiết kế tuân thủ nghiêm ngặt theo kiến trúc MVC, giúp tách bạch rõ ràng giữa phần xử lý dữ liệu và phần hiển thị. Giao diện đồ họa (GUI) được xây dựng bằng thư viện Java Swing mang lại trải nghiệm tương tác thân thiện, trực quan với các biểu mẫu động tự động thay đổi theo đối tượng nhập liệu. Bên cạnh đó, các cơ chế kiểm thử tính hợp lệ của dữ liệu (Validation) và kỹ thuật tuần tự hóa đối tượng (Object Serialization) để đọc/ghi file nhị phân đã giúp hệ thống hoạt động ổn định, bảo toàn được cơ sở dữ liệu sau mỗi phiên làm việc mà không xảy ra hiện tượng thất thoát thông tin.

4.2 Hướng phát triển của phần mềm

Mặc dù hệ thống hiện tại đã đáp ứng tốt các yêu cầu quản lý nghiệp vụ cơ bản, nhưng để phần mềm có thể triển khai thực tế ở quy mô toàn trường Đại học Phenikaa với hàng nghìn cán bộ, giảng viên, một số hạng mục nâng cấp là cần thiết. Định hướng phát triển đầu tiên là chuyển đổi cơ chế lưu trữ từ tệp tin nhị phân (.dat) sang Hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) như MySQL hoặc SQL Server thông qua kết nối JDBC. Việc này sẽ giúp tăng tốc độ truy vấn, xử lý dữ liệu đồng thời và cho phép nhiều chuyên viên nhân sự thao tác trên hệ thống cùng lúc qua mạng LAN.

Thứ hai, hệ thống cần được bổ sung module báo cáo nâng cao, cho phép xuất trực tiếp danh sách nhân sự và bảng lương tổng hợp ra các định dạng tài liệu phổ biến như Excel (.xlsx) hoặc PDF để phục vụ công tác lưu trữ bản cứng và báo cáo lên Ban Giám hiệu. Thứ ba, cơ chế bảo mật cần được mở rộng thành phân quyền nhiều lớp (Role-based access control), trong đó quản trị viên có toàn quyền thay đổi dữ liệu, còn giảng viên/nhân viên có thể được cấp tài khoản riêng chỉ để xem thông tin cá nhân và phiếu lương của mình. Cuối cùng,

trong tầm nhìn dài hạn, lõi xử lý nghiệp vụ (Controller và Model) của phần mềm hoàn toàn có thể được đóng gói lại để làm nền tảng phát triển lên một phiên bản ứng dụng Web (Web Application), giúp đội ngũ nhân sự dễ dàng truy cập và quản lý mọi lúc, mọi nơi thông qua trình duyệt internet.

Tổng kết

Qua quá trình nghiên cứu và thực hiện đề tài "Xây dựng phần mềm quản lý nhân sự Đại học Phenikaa", nhóm thực hiện đã hoàn thành mục tiêu xây dựng một ứng dụng hoàn chỉnh, đáp ứng các yêu cầu cơ bản của bài toán quản lý nhân sự trong môi trường giáo dục đại học. Đồ án không chỉ là sản phẩm phần mềm vận hành ổn định mà còn là minh chứng cho việc áp dụng hiệu quả các nguyên lý Lập trình hướng đối tượng (OOP) vào thực tiễn.

Về mặt kỹ thuật, đề tài đã thể hiện được sự làm chủ công nghệ thông qua việc thiết kế hệ thống theo mô hình kiến trúc phân lớp giúp mã nguồn rõ ràng và dễ bảo trì. Vận dụng thành thạo các cấu trúc dữ liệu (Collection Framework) và thuật toán xử lý danh sách. Xây dựng giao diện người dùng trực quan bằng thư viện Java Swing. Thực hiện cơ chế lưu trữ dữ liệu bền vững thông qua kỹ thuật Serialization (File I/O).

Về mặt nghiệp vụ, phần mềm đã giải quyết thành công bài toán đặc thù của Đại học Phenikaa. Cụ thể là việc phân loại đối tượng nhân sự (Giảng viên và Khối hành chính), áp dụng tính đa hình để xử lý logic tính lương phức tạp, và đặc biệt là cơ chế tính thưởng nghiên cứu khoa học thông qua Interface – một yêu cầu thực tế quan trọng trong môi trường đại học định hướng nghiên cứu.

Tuy nhiên, do giới hạn về thời gian thực hiện cũng như kinh nghiệm thực tiễn, sản phẩm vẫn còn một số hạn chế nhất định về tối ưu hóa hiệu năng và trải nghiệm người dùng (UX). Nhóm thực hiện nhận thức rõ những điểm cần cải thiện và định hướng sẽ phát triển thêm các tính năng nâng cao (như kết nối cơ sở dữ liệu SQL, phân quyền người dùng chi tiết) trong tương lai.

Em thực hiện xin gửi lời cảm ơn chân thành đến Thầy đã tận tình hướng dẫn và cung cấp những kiến thức nền tảng quý báu trong suốt quá trình học tập. Em rất mong nhận được những ý kiến đóng góp, đánh giá từ Thầy để sản phẩm được hoàn thiện hơn, đồng thời rút ra những bài học kinh nghiệm cho các dự án phần mềm tiếp theo.

Tài liệu tham khảo

- [1] Herbert Schildt, *Java: The Complete Reference, Twelfth Edition*, McGraw Hill, 2021.
- [2] Bruce Eckel (2006), *Thinking in Java (4th Edition)*, Nhà xuất bản Prentice Hall.
- [3] Kathy Sierra, Bert Bates (2005), *Head First Java (2nd Edition)*, O'Reilly Media.
- [4] Oracle Corporation, *Java Platform, Standard Edition 8 API Specification*
- [5] Oracle Corporation, *Creating a GUI With Swing (The Java™ Tutorials)*.
- [6] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides (1994), *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley.

Phụ lục: Source Code

A. Khởi MODEL

File 1: NghienCuuKhoaHoc.java (Interface)

```
package model;

public interface NghienCuuKhoaHoc {
    double tinhThuongNghienCuu();
}
```

File 2: CanBoPhenikaa.java (Abstract Class - Lớp cha)

```
package model;

import java.io.Serializable;

public abstract class CanBoPhenikaa implements Serializable {
    private static final long serialVersionUID = 1L;
    protected String maCB;
    protected String hoTen;
    protected int namSinh;
    protected double heSoLuong;
    protected final double LUONG_CO_SO = 1800000;

    public CanBoPhenikaa(String maCB, String hoTen, int namSinh, double heSoLuong) {
        this.maCB = maCB;
        this.hoTen = hoTen;
        this.namSinh = namSinh;
    }
}
```

```
    this.heSoLuong = heSoLuong;
}
```

```
public String getMaCB() { return maCB; }
public void setMaCB(String maCB) { this.maCB = maCB; }
```

```
public String getHoTen() { return hoTen; }
public void setHoTen(String hoTen) { this.hoTen = hoTen; }
```

```
public int getNamSinh() { return namSinh; }
public void setNamSinh(int namSinh) { this.namSinh = namSinh; }
```

```
public double getHeSoLuong() { return heSoLuong; }
public void setHeSoLuong(double heSoLuong) { this.heSoLuong = heSoLuong; }
```

```
public abstract double tinhLuong();
public abstract String getDonViCongTac();
}
```

File 3: GiangVien.java (Lớp con)

```
package model;
```

```
public class GiangVien extends CanBoPhenikaa implements NghienCuuKhoaHoc {
    private final String khoa;
    private final String trinhDo;
    private final int soTietDay;
    private final int soBaiBaoISI;
```



```

public GiangVien(String maCB, String hoTen, int namSinh, double heSoLuong,
    String khoa, String trinhDo, int soTietDay, int soBaiBaoISI) {
    super(maCB, hoTen, namSinh, heSoLuong);
    this.khoa = khoa;
    this.trinhDo = trinhDo;
    this.soTietDay = soTietDay;
    this.soBaiBaoISI = soBaiBaoISI;
}

```

```

@Override
public double tinhThuongNghienCuu() {
    return this.soBaiBaoISI * 5000000;
}

```

```

@Override
public double tinhLuong() {
    double phuCapTrinhDo = trinhDo.equalsIgnoreCase("Tiến sĩ") ? 3000000 : 1000000;
    double luongGiangDay = soTietDay * 120000;
    return (heSoLuong * LUONG_CO_SO) + luongGiangDay + phuCapTrinhDo +
    tinhThuongNghienCuu();
}

```

```

@Override
public String getDonViCongTac() {
    return "Khoa: " + this.khoa;
}

```

```

    public String getKhoa() { return khoa; }
    public String getTrinhDo() { return trinhDo; }
    public int getSoTietDay() { return soTietDay; }
    public int getSoBaiBaoISI() { return soBaiBaoISI; }
}

package model;

public class NhanVienHC extends CanBoPhenikaa {
    private final String phongBan;
    private final String chucVu;
    private final int soNgayCong;

    public NhanVienHC(String maCB, String hoTen, int namSinh, double heSoLuong,
        String phongBan, String chucVu, int soNgayCong) {
        super(maCB, hoTen, namSinh, heSoLuong);
        this.phongBan = phongBan;
        this.chucVu = chucVu;
        this.soNgayCong = soNgayCong;
    }

    @Override
    public double tinhLuong() {
        double phuCapChucVu = chucVu.equalsIgnoreCase("Trưởng phòng") ? 4000000 :
1000000;

        double luongNgayCong = soNgayCong * 250000;
        return (heSoLuong * LUONG_CO_SO) + luongNgayCong + phuCapChucVu;
    }
}

```

```

@Override
public String getDonViCongTac() {
    return "Phòng: " + this.phongBan;
}

public String getPhongBan() { return phongBan; }
public String getChucVu() { return chucVu; }
public int getSoNgayCong() { return soNgayCong; }
}

```

B. Khởi VIEW

```

package view;

import javax.swing.*.*;
import java.awt.*.*;

public class Login extends JFrame {
    private JTextField txtUsername;
    private JPasswordField txtPassword;
    private final JButton btnLogin;
    private final JButton btnExit;

    public Login() {
        setTitle("Đăng nhập - QLNS Đại học Phenikaa");
        setSize(400, 220);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

```

```
setLocationRelativeTo(null);  
setLayout(new BorderLayout());
```

```
JPanel panelInput = new JPanel(new GridLayout(2, 2, 10, 10));  
panelInput.setBorder(BorderFactory.createEmptyBorder(15, 30, 15, 30));  
panelInput.add(new JLabel("Tài khoản:"));  
txtUsername = new JTextField();  
panelInput.add(txtUsername);  
panelInput.add(new JLabel("Mật khẩu:"));  
txtPassword = new JPasswordField();  
panelInput.add(txtPassword);
```

```
JPanel panelButtons = new JPanel(new FlowLayout());  
panelButtons.setBorder(BorderFactory.createEmptyBorder(0, 0, 10, 0));  
btnLogin = new JButton("Đăng nhập");  
btnExit = new JButton("Thoát");  
panelButtons.add(btnLogin);  
panelButtons.add(btnExit);
```

```
JLabel lblTitle = new JLabel("HỆ THỐNG QUẢN LÝ NHÂN SỰ",  
SwingConstants.CENTER);
```

```
lblTitle.setFont(new Font("Arial", Font.BOLD, 15));  
lblTitle.setBorder(BorderFactory.createEmptyBorder(15, 0, 0, 0));
```

```
add(lblTitle, BorderLayout.NORTH);  
add(panelInput, BorderLayout.CENTER);  
add(panelButtons, BorderLayout.SOUTH);
```

```

        btnLogin.addActionListener(e -> {
            String user = txtUsername.getText();
            String pass = new String(txtPassword.getPassword());
            if (user.equals("admin") && pass.equals("123")) {
                JOptionPane.showMessageDialog(null, "Đăng nhập thành công!");
                dispose();
                new MainMenu().setVisible(true);
            } else {
                JOptionPane.showMessageDialog(null, "Sai tài khoản hoặc mật khẩu!", "Lỗi",
JOptionPane.ERROR_MESSAGE);
            }
        });

        btnExit.addActionListener(e -> System.exit(0));
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Login().setVisible(true));
    }
}

```

File 6: MainMenu.java

```

package view;

```

```

import javax.swing.*;
import java.awt.*;

```

```

public class MainMenu extends JFrame {

    public MainMenu() {
        setTitle("Hệ thống Quản lý Nhân sự Phenikaa");
        setSize(450, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        JLabel lblTitle = new JLabel("CHỌN PHÂN HỆ QUẢN LÝ",
        SwingConstants.CENTER);
        lblTitle.setFont(new Font("Arial", Font.BOLD, 22));
        lblTitle.setBorder(BorderFactory.createEmptyBorder(20, 0, 20, 0));
        add(lblTitle, BorderLayout.NORTH);

        JPanel pnlButtons = new JPanel(new GridLayout(2, 1, 15, 15));
        pnlButtons.setBorder(BorderFactory.createEmptyBorder(10, 50, 20, 50));

        JButton btnGiangVien = new JButton("Quản lý Giảng viên");
        btnGiangVien.setFont(new Font("Arial", Font.BOLD, 16));

        JButton btnNhanVien = new JButton("Quản lý Nhân viên Hành chính");
        btnNhanVien.setFont(new Font("Arial", Font.BOLD, 16));

        pnlButtons.add(btnGiangVien);
        pnlButtons.add(btnNhanVien);
        add(pnlButtons, BorderLayout.CENTER);
    }
}

```

```

JPanel pnlBottom = new JPanel();
JButton btnLogout = new JButton("Đăng xuất");
pnlBottom.add(btnLogout);
add(pnlBottom, BorderLayout.SOUTH);

btnGiangVien.addActionListener(e -> {
    new HRManagementGUI("Giảng viên").setVisible(true);
    dispose();
});

btnNhanVien.addActionListener(e -> {
    new HRManagementGUI("Nhân viên Hành chính").setVisible(true);
    dispose();
});

btnLogout.addActionListener(e -> {
    System.exit(0);
});
}
}

```

File 7: HRManagementGUI.java

```
package view;
```

```

import controller.QuanLyCanBo;
import model.CanBoPhenikaa;
import model.GiangVien;

```

```

import model.NhanVienHC;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.io.File;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class HRManagementGUI extends JFrame {
    private QuanLyCanBo quanLy;
    private DefaultTableModel tableModel;
    private JTable table;

    private String loaiQuanLy;
    private List<CanBoPhenikaa> danhSachHienTai;

    private JTextField txtMaCB, txtHoTen, txtNamSinh, txtHeSoLuong;
    private JComboBox<String> cmbLoaiCB;

    private JPanel panelGV;
    private JTextField txtKhoa, txtSoTiet, txtSoBaiBao;
    private JComboBox<String> cmbTrinhDoGV;

    private JPanel panelNV;

```



```

private JTextField txtPhongBan, txtSoNgayCong;
private JComboBox<String> cmbChucVuNV;

private JPanel panelDynamic;
private CardLayout cardLayout;

public HRManagementGUI(String loaiQuanLy) {
    this.loaiQuanLy = loaiQuanLy;
    this.danhSachHienTai = new ArrayList<>();
    quanLy = new QuanLyCanBo();

    try {
        quanLy.docFile("NhanSuPhenikaa.dat");
    } catch (Exception e) {
    }

    setTitle("Quản lý " + loaiQuanLy + " - Đại học Phenikaa");
    setSize(1050, 700);
    setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout(10, 10));

    add(taoPanelNhapLieu(), BorderLayout.NORTH);
    add(taoPanelBangDuLieu(), BorderLayout.CENTER);
    add(taoPanelChucNang(), BorderLayout.SOUTH);

    capNhatBang();
}

```

```

addWindowListener(new WindowAdapter() {
    @Override
    public void windowClosing(WindowEvent e) {
        thoatUngDung();
    }
});
}

```

```

private JPanel taoPanelNhapLieu() {
    JPanel pnlTop = new JPanel();
    pnlTop.setLayout(new BoxLayout(pnlTop, BoxLayout.Y_AXIS));
    pnlTop.setBorder(BorderFactory.createTitledBorder("Thông tin chi tiết hồ sơ nhân sự"));
}

```

```

JPanel pnlChung = new JPanel(new GridLayout(0, 4, 20, 15));
pnlChung.setBorder(BorderFactory.createEmptyBorder(10, 15, 5, 15));
pnlChung.add(new JLabel("Mã Cán bộ:")); txtMaCB = new JTextField();
pnlChung.add(txtMaCB);

pnlChung.add(new JLabel("Họ và Tên:")); txtHoTen = new JTextField();
pnlChung.add(txtHoTen);

pnlChung.add(new JLabel("Năm sinh:")); txtNamSinh = new JTextField();
pnlChung.add(txtNamSinh);

pnlChung.add(new JLabel("Hệ số lương:")); txtHeSoLuong = new JTextField();
pnlChung.add(txtHeSoLuong);

```

```

JPanel pnlLoai = new JPanel(new FlowLayout(FlowLayout.LEFT, 15, 10));
pnlLoai.add(new JLabel("Loại cán bộ: "));

```

```

        cmbLoaiCB = new JComboBox<>(new String[]{"Giảng viên", "Nhân viên Hành
chính"});

        cmbLoaiCB.setSelectedItem(loaiQuanLy);

        cmbLoaiCB.setEnabled(false);

        pnlLoai.add(cmbLoaiCB);


        cardLayout = new CardLayout();

        panelDynamic = new JPanel(cardLayout);

        panelDynamic.setBorder(BorderFactory.createEmptyBorder(0, 15, 15, 15));


        panelGV = new JPanel(new GridLayout(0, 4, 20, 15));

        panelGV.add(new JLabel("Khoa công tác:")); txtKhoa = new JTextField();
        panelGV.add(txtKhoa);

        panelGV.add(new JLabel("Trình độ:")); cmbTrinhDoGV = new JComboBox<>(new
String[]{"Thạc sĩ", "Tiến sĩ"}); panelGV.add(cmbTrinhDoGV);

        panelGV.add(new JLabel("Số tiết dạy:")); txtSoTiet = new JTextField();
        panelGV.add(txtSoTiet);

        panelGV.add(new JLabel("Số bài ISI:")); txtSoBaiBao = new JTextField();
        panelGV.add(txtSoBaiBao);


        panelNV = new JPanel(new GridLayout(0, 4, 20, 15));

        panelNV.add(new JLabel("Phòng ban:")); txtPhongBan = new JTextField();
        panelNV.add(txtPhongBan);

        panelNV.add(new JLabel("Chức vụ:")); cmbChucVuNV = new JComboBox<>(new
String[]{"Chuyên viên", "Phó phòng", "Trưởng phòng"}); panelNV.add(cmbChucVuNV);

        panelNV.add(new JLabel("Số ngày công:")); txtSoNgayCong = new JTextField();
        panelNV.add(txtSoNgayCong);

        panelNV.add(new JLabel("")); panelNV.add(new JLabel(""));

```

```

panelDynamic.add(panelGV, "Giảng viên");
panelDynamic.add(panelNV, "Nhân viên Hành chính");
cardLayout.show(panelDynamic, loaiQuanLy);

pnlTop.add(pnlChung);
pnlTop.add(pnlLoai);
pnlTop.add(panelDynamic);

return pnlTop;
}

private JPanel taoPanelBangDuLieu() {
    JPanel pnlCenter = new JPanel(new BorderLayout());
    pnlCenter.setBorder(BorderFactory.createTitledBorder("Danh sách " +
loaiQuanLy));

    String[] cols = {"Mã CB", "Họ Tên", "Năm sinh", "Phân loại", "Đơn vị/Phòng", "Tổng
Lương (VNĐ)"};
    tableModel = new DefaultTableModel(cols, 0);
    table = new JTable(tableModel);
    pnlCenter.add(new JScrollPane(table), BorderLayout.CENTER);

    table.addMouseListener(new MouseAdapter() {
        @Override
        public void mouseClicked(MouseEvent e) {
            int row = table.getSelectedRow();
            if (row >= 0) hienThiChiTietLenForm(row);
        }
    });
}

```

```

    }
});

return pnlCenter;
}

private JPanel taoPanelChucNang() {
    JPanel pnlSouth = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 15));

    JButton btnQuayLai = new JButton("Quay lại Menu");
    JButton btnThem = new JButton("Thêm mới");
    JButton btnSua = new JButton("Cập nhật");
    JButton btnXoa = new JButton("Xóa");
    JButton btnTim = new JButton("Tìm kiếm");
    JButton btnSapXep = new JButton("Sắp xếp lương");
    JButton btnNhap = new JButton("Nhập File");
    JButton btnLuu = new JButton("Xuất File");
    JButton btnTongLuong = new JButton("Thống kê");
    JButton btnLamMoi = new JButton("Làm mới");

    pnlSouth.add(btnQuayLai);
    pnlSouth.add(btnThem); pnlSouth.add(btnSua); pnlSouth.add(btnXoa);
    pnlSouth.add(btnTim); pnlSouth.add(btnSapXep);
    pnlSouth.add(btnNhap); pnlSouth.add(btnLuu);
    pnlSouth.add(btnTongLuong); pnlSouth.add(btnLamMoi);

    btnQuayLai.addActionListener(e -> {

```

```

try { quanLy.luuFile("NhanSuPhenikaa.dat"); } catch (Exception ex) {}

new MainMenu().setVisible(true);

dispose();

});

btnThem.addActionListener(e -> xuLyThem());
btnSua.addActionListener(e -> xuLySua());
btnXoa.addActionListener(e -> xuLyXoa());
btnNhap.addActionListener(e -> xuLyDocFile());
btnLuu.addActionListener(e -> xuLyLuuFile());
btnLamMoi.addActionListener(e -> { xoaTrangForm(); capNhatBang(); });

btnTongLuong.addActionListener(e -> {
    double tong = 0;
    for (CanBoPhenikaa cb : danhSachHienTai) {
        tong += cb.tinhLuong();
    }
    JOptionPane.showMessageDialog(this,
        String.format("Tổng quỹ lương khối %s: %,0f VNĐ", loaiQuanLy, tong),
        "Thông kê", JOptionPane.INFORMATION_MESSAGE);
});

btnTim.addActionListener(e -> {
    String tuKhoa = JOptionPane.showInputDialog(this, "Nhập mã " + loaiQuanLy +
" cần tìm:");
    if (tuKhoa != null && !tuKhoa.trim().isEmpty()) {
        List<CanBoPhenikaa> listTim = quanLy.timKiemTheoMa(tuKhoa);

```

```

List<CanBoPhenikaa> ketQuaLoc = new ArrayList<>();
for (CanBoPhenikaa cb : listTim) {
    if (loaiQuanLy.equals("Giảng viên") && cb instanceof GiangVien)
ketQuaLoc.add(cb);
    else if (loaiQuanLy.equals("Nhân viên Hành chính") && cb instanceof
NhanVienHC) ketQuaLoc.add(cb);
}

if (ketQuaLoc.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Không tìm thấy hồ sơ phù hợp!");
} else {
    hienThiDanhSach(ketQuaLoc);
}
} else {
    capNhatBang();
}
});

btnSapXep.addActionListener(e -> {
    quanLy.sapXepTheoLuong();
    capNhatBang();
    JOptionPane.showMessageDialog(this, "Đã sắp xếp theo lương giảm dần!");
});

return pnlSouth;
}

```

```

private CanBoPhenikaa layDuLieuTuForm() throws NumberFormatException {
    String ma = txtMaCB.getText();
    String ten = txtHoTen.getText();
    if(ma.isEmpty() || ten.isEmpty()) throw new NumberFormatException("Thiếu thông tin");
    int nam = Integer.parseInt(txtNamSinh.getText());
    double heSo = Double.parseDouble(txtHeSoLuong.getText());

    if (loaiQuanLy.equals("Giảng viên")) {
        String khoa = txtKhoa.getText();
        String trinhDo = (String) cmbTrinhDoGV.getSelectedItem();
        int tiet = Integer.parseInt(txtSoTiet.getText());
        int baiBao = Integer.parseInt(txtSoBaiBao.getText());
        return new GiangVien(ma, ten, nam, heSo, khoa, trinhDo, tiet, baiBao);
    } else {
        String phong = txtPhongBan.getText();
        String chucVu = (String) cmbChucVuNV.getSelectedItem();
        int ngay = Integer.parseInt(txtSoNgayCong.getText());
        return new NhanVienHC(ma, ten, nam, heSo, phong, chucVu, ngay);
    }
}

private void xuLyThem() {
    try {
        if (quanLy.kiemTraTonTai(txtMaCB.getText())) {
            JOptionPane.showMessageDialog(this, "Mã này đã tồn tại!", "Lỗi",
JOptionPane.ERROR_MESSAGE);

```



```

        return;
    }
    CanBoPhenikaa cb = layDuLieuTuForm();
    quanLy.themCanBo(cb);
    capNhatBang();
    xoaTrangForm();
    JOptionPane.showMessageDialog(this, "Thêm thành công!");
} catch (HeadlessException | NumberFormatException ex) {
    JOptionPane.showMessageDialog(this, "Lỗi nhập liệu! Kiểm tra lại thông tin.",
    "Lỗi", JOptionPane.ERROR_MESSAGE);
}
}

```

```

private void xuLySua() {
    int row = table.getSelectedRow();
    if (row == -1) {
        JOptionPane.showMessageDialog(this, "Vui lòng chọn dòng cần sửa!");
        return;
    }
    try {
        CanBoPhenikaa cbCu = danhSachHienTai.get(row);
        int originalIndex = quanLy.layDanhSach().indexOf(cbCu);

        CanBoPhenikaa cbMoi = layDuLieuTuForm();
        quanLy.capNhatCanBo(originalIndex, cbMoi);
        capNhatBang();
        JOptionPane.showMessageDialog(this, "Cập nhật thành công!");
    }
}

```

```

    } catch (HeadlessException | NumberFormatException ex) {
        JOptionPane.showMessageDialog(this, "Lỗi nhập liệu khi sửa!", "Lỗi",
JOptionPane.ERROR_MESSAGE);
    }
}

```

```

private void xuLyXoa() {
    int row = table.getSelectedRow();
    if (row == -1) {
        JOptionPane.showMessageDialog(this, "Chọn dòng để xóa!");
        return;
    }

    int confirm = JOptionPane.showConfirmDialog(this, "Chắc chắn xóa?", "Xác nhận",
JOptionPane.YES_NO_OPTION);
    if (confirm == JOptionPane.YES_OPTION) {
        CanBoPhenikaa cbCu = danhSachHienTai.get(row);
        int originalIndex = quanLy.layDanhSach().indexOf(cbCu);
        quanLy.xoaCanBo(originalIndex);

        capNhatBang();
        xoaTrangForm();
    }
}

```

```

private void hienThiDanhSach(List<CanBoPhenikaa> list) {
    this.danhSachHienTai = list;
    tableModel.setRowCount(0);
}

```

```

for (CanBoPhenikaa cb : list) {
    String loai = (cb instanceof GiangVien) ? "Giảng viên" : "Nhân viên HC";
    tableModel.addRow(new Object[]{
        cb.getMaCB(), cb.getHoTen(), cb.getNamSinh(),
        loai, cb.getDonViCongTac(),
        String.format("%.0f", cb.tinhLuong())
    });
}
}

private void capNhatBang() {
    List<CanBoPhenikaa> locDanhSach = new ArrayList<>();
    for (CanBoPhenikaa cb : quanLy.layDanhSach()) {
        if (loaiQuanLy.equals("Giảng viên") && cb instanceof GiangVien) {
            locDanhSach.add(cb);
        } else if (loaiQuanLy.equals("Nhân viên Hành chính") && cb instanceof
NhanVienHC) {
            locDanhSach.add(cb);
        }
    }
    hienThiDanhSach(locDanhSach);
}

private void hienThiChiTietLenForm(int index) {
    if (index < 0 || index >= danhSachHienTai.size()) return;

    CanBoPhenikaa cb = danhSachHienTai.get(index);

```

```

txtMaCB.setText(cb.getMaCB());
txtHoTen.setText(cb.getHoTen());
txtNamSinh.setText(String.valueOf(cb.getNamSinh()));
txtHeSoLuong.setText(String.valueOf(cb.getHeSoLuong()));

if (cb instanceof GiangVien giangVien) {
    GiangVien gv = giangVien;
    txtKhoa.setText(gv.getKhoa());
    cmbTrinhDoGV.setSelectedItem(gv.getTrinhDo());
    txtSoTiet.setText(String.valueOf(gv.getSoTietDay()));
    txtSoBaiBao.setText(String.valueOf(gv.getSoBaiBaoISI()));
} else {
    NhanVienHC nv = (NhanVienHC) cb;
    txtPhongBan.setText(nv.getPhongBan());
    cmbChucVuNV.setSelectedItem(nv.getChucVu());
    txtSoNgayCong.setText(String.valueOf(nv.getSoNgayCong()));
}
}

private void xuLyDocFile() {
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setDialogTitle("Chọn file dữ liệu để nhập (.dat)");

    int userSelection = fileChooser.showOpenDialog(this);
    if (userSelection == JFileChooser.APPROVE_OPTION) {
        File fileToRead = fileChooser.getSelectedFile();
        try {

```

```

        quanLy.docFile(fileToRead.getAbsolutePath());
        capNhatBang();
        JOptionPane.showMessageDialog(this, "Tải dữ liệu từ file thành công!");
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Lỗi khi đọc file!\nChi tiết: " +
ex.getMessage(), "Lỗi Nhập File", JOptionPane.ERROR_MESSAGE);
    }
}
}
}

```

```

private void xuLyLuuFile() {
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setDialogTitle("Chọn nơi lưu file dữ liệu (.dat)");

    int userSelection = fileChooser.showSaveDialog(this);
    if (userSelection == JFileChooser.APPROVE_OPTION) {
        File fileToSave = fileChooser.getSelectedFile();
        String filePath = fileToSave.getAbsolutePath();

        if (!filePath.toLowerCase().endsWith(".dat")) {
            filePath += ".dat";
        }

        try {
            quanLy.luuFile(filePath);
            JOptionPane.showMessageDialog(this, "Lưu file thành công tại:\n" + filePath);
        } catch (Exception ex) {

```

```
        JOptionPane.showMessageDialog(this, "Lỗi lưu file: " + ex.getMessage(), "Lỗi  
Xuất File", JOptionPane.ERROR_MESSAGE);
```

```
    }  
}  
}
```

```
private void xoaTrangForm() {  
    txtMaCB.setText(""); txtHoTen.setText("");  
    txtNamSinh.setText(""); txtHeSoLuong.setText("");  
    txtKhoa.setText(""); txtSoTiet.setText(""); txtSoBaiBao.setText("");  
    txtPhongBan.setText(""); txtSoNgayCong.setText("");  
    table.clearSelection();  
}
```

```
private void thoatUngDung() {  
    int confirm = JOptionPane.showConfirmDialog(this, "Bạn có muốn lưu dữ liệu hệ  
thống trước khi thoát?", "Xác nhận", JOptionPane.YES_NO_CANCEL_OPTION);  
    if (confirm == JOptionPane.YES_OPTION) {  
        try {  
            quanLy.luuFile("NhanSuPhenikaa.dat");  
        } catch (Exception e) {}  
        System.exit(0);  
    } else if (confirm == JOptionPane.NO_OPTION) {  
        System.exit(0);  
    }  
}  
}}
```

C. Khởi CONTROLLER

File 8: QuanLyCanBo.java

```
package controller;

import model.CanBoPhenikaa;
import java.io.*;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class QuanLyCanBo {
    private List<CanBoPhenikaa> danhSach;

    public QuanLyCanBo() {
        danhSach = new ArrayList<>();
    }

    public void themCanBo(CanBoPhenikaa cb) {
        danhSach.add(cb);
    }

    public void xoaCanBo(int index) {
        if (index >= 0 && index < danhSach.size()) {
            danhSach.remove(index);
        }
    }
}
```

```

public void capNhatCanBo(int index, CanBoPhenikaa cbMoi) {
    if (index >= 0 && index < danhSach.size()) {
        danhSach.set(index, cbMoi);
    }
}

```

```

public List<CanBoPhenikaa> layDanhSach() {
    return danhSach;
}

```

```

public boolean kiemTraTonTai(String maCB) {
    for (CanBoPhenikaa cb : danhSach) {
        if (cb.getMaCB().equalsIgnoreCase(maCB)) {
            return true;
        }
    }
    return false;
}

```

```

public List<CanBoPhenikaa> timKiemTheoMa(String maCB) {
    List<CanBoPhenikaa> ketQua = new ArrayList<>();
    for (CanBoPhenikaa cb : danhSach) {
        if (cb.getMaCB().equalsIgnoreCase(maCB.trim())) {
            ketQua.add(cb);
        }
    }
    return ketQua;
}

```



```
}
```

```
public void sapXepTheoLuong() {  
    Collections.sort(danhSach, (CanBoPhenikaa cb1, CanBoPhenikaa cb2) ->  
        Double.compare(cb2.tinhLuong(), cb1.tinhLuong()));  
}
```

```
public double tinhTongQuyLuong() {  
    double tong = 0;  
    for (CanBoPhenikaa cb : danhSach) {  
        tong += cb.tinhLuong();  
    }  
    return tong;  
}
```

```
public void luuFile(String fileName) throws IOException {  
    try (ObjectOutputStream oos = new ObjectOutputStream(new  
        FileOutputStream(fileName))) {  
        oos.writeObject(danhSach);  
    }  
}
```

```
public void taoDuLieuMau() {  
    danhSach.clear();
```

```
    danhSach.add(new model.GiangVien("GV001", "Nguyễn Văn An", 1980, 3.5,  
        "CNTT", "Tiến sĩ", 120, 3));
```

```
        danhSach.add(new model.GiangVien("GV002", "Trần Thị Bích", 1985, 2.8, "Kinh  
Tế", "Thạc sĩ", 80, 1));
```

```
        danhSach.add(new model.NhanVienHC("NV001", "Lê Hoàng", 1990, 2.34, "Phòng  
Đào tạo", "Chuyên viên", 24));
```

```
        danhSach.add(new model.NhanVienHC("NV002", "Phạm Văn Dũng", 1978, 4.0,  
"Phòng Tổ chức", "Trưởng phòng", 26));
```

```
    }
```

```
@SuppressWarnings("unchecked")
```

```
public void docFile(String fileName) throws IOException, ClassNotFoundException {
```

```
    File f = new File(fileName);
```

```
    if(!f.exists()) return;
```

```
    try        (ObjectInputStream    ois        =        new        ObjectInputStream(new  
FileInputStream(fileName))) {
```

```
        danhSach = (List<CanBoPhenikaa>) ois.readObject();
```

```
    }
```

```
}
```

```
}
```